Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Foundations of Programming

Assignment No. : 5

PROBLEM STATEMENT:
Write a program in C to perform basic matrix operations such as:
1. Addition of two matrices
2. Saddle point of a matrix
3. Inverse of a matrix
4. Magic square of a matrix

OBJECTIVE:

- To understand matrix representation in C using two-dimensional arrays.
- To implement various matrix operations using C programming.
- To develop logical thinking for solving matrix-based problems.

THEORY:

1) Addition of two matrices

Add corresponding elements.

Example:

A = $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$,

B = $\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

A + B = $\begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$

---

2) Saddle point of a matrix

Element that is minimum in its row and maximum in its column.

Example:

$$\begin{bmatrix} 3 & 1 & 3 \\ 8 & 5 & 6 \\ 7 & 2 & 9 \end{bmatrix}$$

Saddle point = 5

---

3) Inverse of a matrix

Exists only if determinant ≠ 0.

Example:

A = $\begin{bmatrix} 4 & 7 \\ 2 & 6 \end{bmatrix}$

|A| = 10

$$A^{-1} = \frac{1}{10}\begin{bmatrix} 6 & -7 \\ -2 & 4 \end{bmatrix} = \begin{bmatrix} 0.6 & -0.7 \\ -0.2 & 0.4 \end{bmatrix}$$

---

4) Magic square

Row sum = Column sum = Diagonal sum.

Example:

$$\begin{bmatrix} 8 & 1 & 6 \\ 3 & 5 & 7 \\ 4 & 9 & 2 \end{bmatrix}$$

All sums = 15

PLATFORM:

Linux

ALGORITHM:

1. Start
2. Read the order of the matrix
3. Read elements of the matrices
4. Perform matrix addition
5. Check for saddle point
6. Find inverse of the matrix (if determinant ≠ 0)
7. Check whether the matrix is a magic square
8. Display results
9. Stop

C PROGRAM: (Type C Program)

```
#include<stdio.h>


void addMatrix();

void saddlePoint();

void inverseMatrix();

void magicSquare();


int main()

{

   int ch;

   do{
```

```
    printf("\n1.Addition\n2.Saddle Point\n3.Inverse\n4.Magic Square\n5.Exit\n");
    printf("Enter choice: ");
    scanf("%d",&ch);

    switch(ch)
    {
      case 1: addMatrix(); break;
      case 2: saddlePoint(); break;
      case 3: inverseMatrix(); break;
      case 4: magicSquare(); break;
    }
  }while(ch!=5);
}

/* 1. ADDITION */
void addMatrix()
{
  int a[10][10],b[10][10],c[10][10],r,c1,i,j;

  printf("Enter rows and cols: ");
  scanf("%d%d",&r,&c1);

  printf("Enter Matrix A:\n");
  for(i=0;i<r;i++)
    for(j=0;j<c1;j++)
      scanf("%d",&a[i][j]);
```

```
    printf("Enter Matrix B:\n");
    for(i=0;i<r;i++)
        for(j=0;j<c1;j++)
            scanf("%d",&b[i][j]);

    for(i=0;i<r;i++)
        for(j=0;j<c1;j++)
            c[i][j]=a[i][j]+b[i][j];

    printf("Result:\n");
    for(i=0;i<r;i++){
        for(j=0;j<c1;j++)
            printf("%d ",c[i][j]);
        printf("\n");
    }
}

/* 2. SADDLE POINT */
void saddlePoint()
{
    int a[10][10],r,c,i,j,min,max,flag=0;

    printf("Enter rows and cols: ");
    scanf("%d%d",&r,&c);
```

```
    printf("Enter matrix:\n");
    for(i=0;i<r;i++)
        for(j=0;j<c;j++)
            scanf("%d",&a[i][j]);

    for(i=0;i<r;i++)
    {
        min=a[i][0];
        int col=0;
        for(j=1;j<c;j++)
            if(a[i][j]<min){min=a[i][j]; col=j;}

        max=1;
        for(j=0;j<r;j++)
            if(a[j][col]>min) max=0;

        if(max){
            printf("Saddle Point = %d\n",min);
            flag=1;
        }
    }
    if(!flag) printf("No Saddle Point\n");
}

/* 3. INVERSE (2x2) */
void inverseMatrix()
```

```
{
   float a[2][2],det;

   printf("Enter 2x2 matrix:\n");
   for(int i=0;i<2;i++)
      for(int j=0;j<2;j++)
         scanf("%f",&a[i][j]);

   det = a[0][0]*a[1][1] - a[0][1]*a[1][0];

   if(det==0){
      printf("Inverse not possible\n");
      return;
   }

   printf("Inverse Matrix:\n");
   printf("%.2f %.2f\n%.2f %.2f\n",
   a[1][1]/det, -a[0][1]/det,
   -a[1][0]/det, a[0][0]/det);
}

/* 4. MAGIC SQUARE CHECK */
void magicSquare()
{
   int a[10][10],n,i,j,sum=0,diag1=0,diag2=0,flag=1;
```

```
printf("Enter order: ");
scanf("%d",&n);

printf("Enter matrix:\n");
for(i=0;i<n;i++)
   for(j=0;j<n;j++)
      scanf("%d",&a[i][j]);

for(j=0;j<n;j++) sum+=a[0][j];

for(i=0;i<n;i++){
   int rsum=0,csum=0;
   for(j=0;j<n;j++){
      rsum+=a[i][j];
      csum+=a[j][i];
   }
   if(rsum!=sum || csum!=sum) flag=0;
   diag1+=a[i][i];
   diag2+=a[i][n-i-1];
}

if(diag1!=sum || diag2!=sum) flag=0;

if(flag) printf("Magic Square\n");
else printf("Not Magic Square\n");
```

}

INPUT:

```
1
Enter rows and cols: 2 2
Enter Matrix A:
1 2
3 4
Enter Matrix B:
5 6
7 8
```

OUTPUT:

```
Result:
6 8
10 12
```

CONCLUSION:

Thus, we have learned how to perform basic matrix operations using C programming and understood their practical applications.

FAQs:

1. What is a saddle point in a matrix?
2. When does a matrix have an inverse?
3. What is a magic square?